



VNIVERSITAT
E VALÈNCIA

Escuela Técnica Superior de Ingeniería
Departamento de Informática

Tema 2

Frameworks Javascript

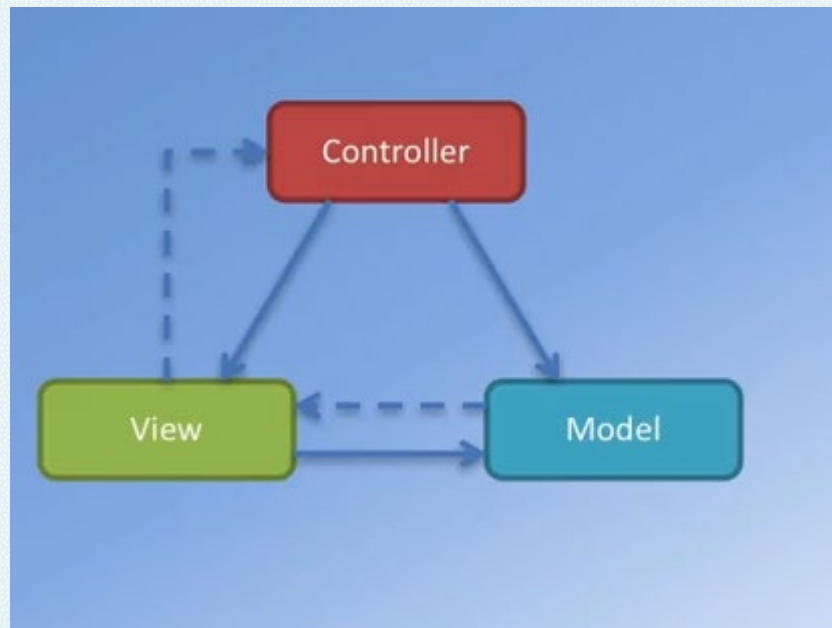
Programación del lado del cliente y visualización
2025-2026

Ricardo Olanda Rodríguez

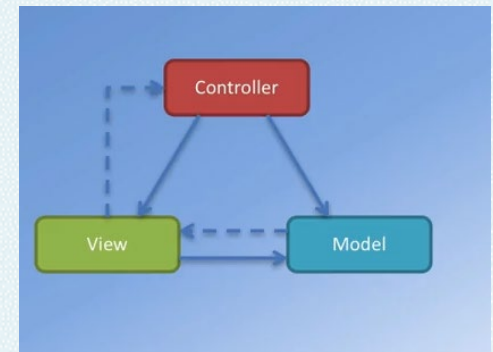
Tema
Tema 1 – Revisión de HTML5, CSS y JavaScript
Tema 2 – Frameworks Javascript
Tema 3 – Comunicación asíncrona y WebSockets
Tema 4 – Almacenamiento en el lado del cliente
Tema 5 – Frameworks para la visualización de datos

Tema 1	Frameworks Javascript	
Objetivos		
Realizar un estudio de los principales frameworks de javascript.		
Contenidos		
1.1 - JSON		
1.2 - JQuery		
1.3 - Angular		

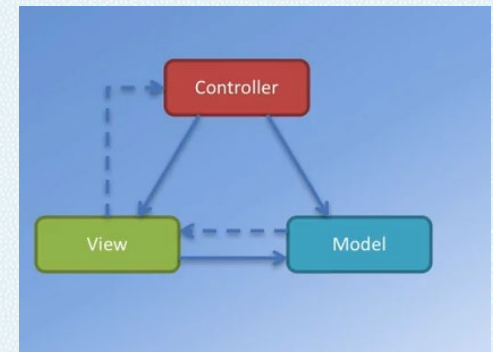
- Vista: Presentar la información al usuario.
- Modelo: Se encarga de la lógica de la aplicación y del estado de la misma en cada momento.
- Controlador: Media entre la vista y el modelo.



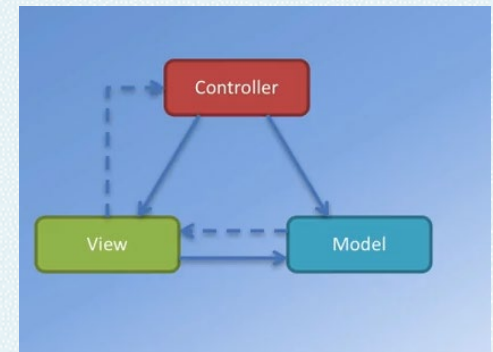
- **Modelo:**
 - Gestiona los datos de la aplicación.
 - Responde a las peticiones de información acerca de su estado.
 - Para la visualización del contenido.
 - Responde a las peticiones de cambio de estado.
 - El controlador normalmente realiza estas peticiones.
 - En un modelo basado en eventos, el modelo notifica cuando la información cambia.
 - Permitiría actualizar la información visualizada.



- Vista:
 - Representación del estado actual del modelo.
 - Presenta la información al usuario, de forma que éste pueda interactuar con ella a través de un interfaz.
 - Pueden existir múltiples vistas para una misma información
 - En función del tamaño de la pantalla del dispositivo: móvil, ordenador, tableta, ...

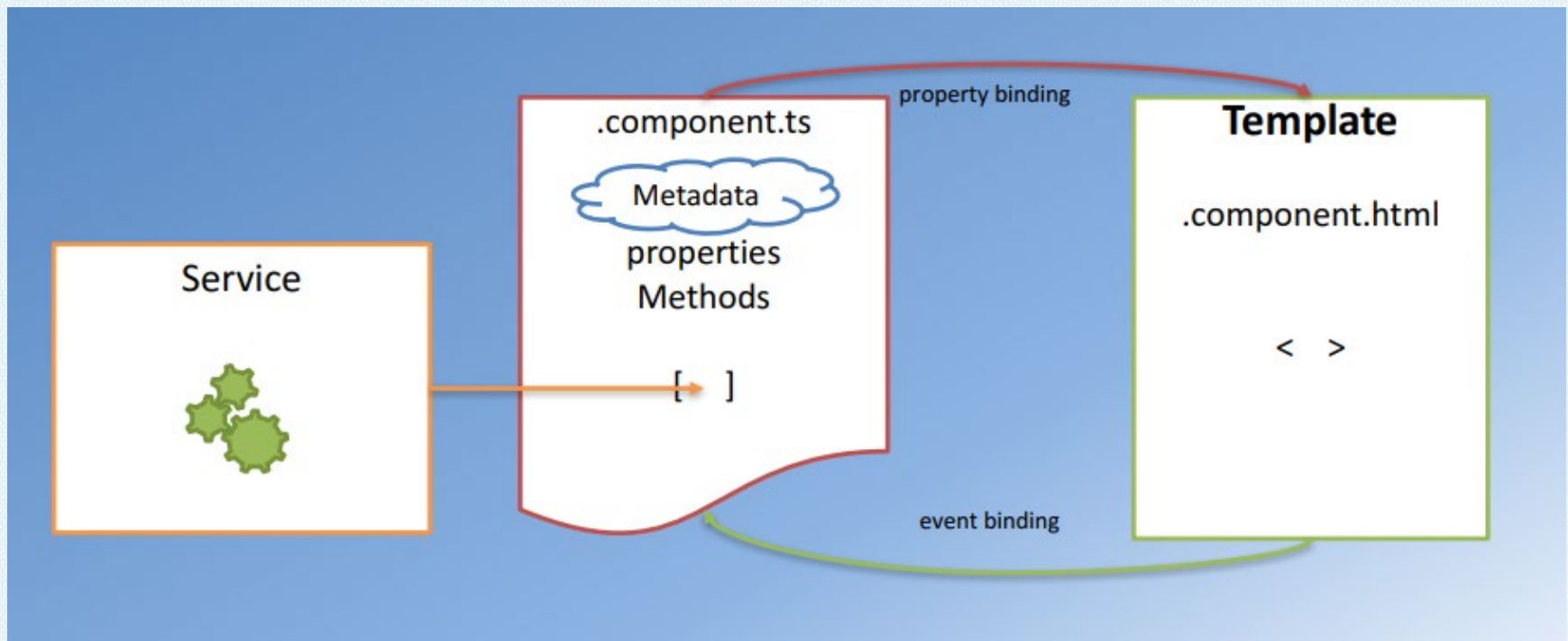


- Controlador:
 - Recibe información proporcionada por el usuario en la vista, y genera una respuesta a esa información recibida realizando peticiones al modelo.
 - El controlador provoca el cambio en el estado del modelo.
 - El controlado puede provocar un cambio en la información que se visualiza en la vista.



- Los componentes delegarán en servicios:
 - La obtención de la información de los servidores.
 - La identificación del usuario (login).
 - La validación de la información proporcionada por el usuario.
- Dividir la lógica de aplicación en diferentes servicios, liberando a los componentes de esa carga:
 - Comunicación con los servidores, validación de los datos, ...
 - “Dependency Injection”

- El servicio proporciona al componente ciertas características que puede utilizar.



- Es un patrón de diseño de software utilizado cuando tienes objetos que dependen de otros objetos.
- “Dependency”: Tenemos un objeto que depende de otro para realizar su tarea.
- “Injection”: Poner a disposición de un objeto, aquel otro objeto del que depende.
 - Para el objeto dependiente, la implementación de ese objeto es indiferente, sólo va a hacer uso de él.

- Tres formas de acceder a los objetos de los que se depende:
 - Crear el objeto del que se depende usando el operador “new”.
 - Ej. Crear un objeto de una clase particular.
 - Utilizar una variable global.
 - Pasando el objeto del que se depende al objeto dependiente cuando se necesite.
 - Flexible: existen diferentes mecanismos para pasar el objeto.
 - El código es más sencillo, no hay que preocuparse de todo el proceso (crear variable, acceder a ella, ...).
 - Las pruebas son más fáciles de realizar.

- El servicio que proporciona un objeto.
- El cliente que va a hacer uso del servicio.
- Interfaz que se va a utilizar para acceder al servicio.
- El Injector, la entidad que se va a encargar de inyectar el objeto del que se depende en el objeto que lo requiere.

- Separa la lógica de negocio y la construcción de la dependencia.
 - Permite definir objetos, los cuales se pueden inyectar allí donde se necesiten.
- Angular dispone de un subsistema de inyección que se encarga de:
 - Crear los servicios
 - Inyectar esos servicios a los componentes cuando los necesiten.

- Creamos un directorio “servicios” dentro del directorio app.
- Ejecutamos la instrucción: “ng generate service servicios/productoServicio”
- Se generan dos ficheros:
 - “producto-servicio.spec.ts”: se utiliza para testear el servicio.
 - “producto-servicio.ts”: contendrá la definición del servicio.

```
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root'
})
export class ProductoServicio {

  constructor() { }

}
```

- Modificar el servicio para que nos devuelva el vector de productos y comentarios.

```
import { Injectable } from '@angular/core';
```

```
import { Producto } from '../compartido/producto';
```

→ Se importan las clases necesarias

```
import { PRODUCTOS } from '../compartido/productos';
```

```
@Injectable()
```

```
export class ProductoServicio {
```

```
  constructor() { }
```

```
  getProductos(): Producto[] {
```

→ Se crea una función que devuelva el vector de productos

```
    return PRODUCTOS;
```

```
  }
```

```
}
```

- Este código hace accesible el servicio al resto de módulos de la aplicación.

```
@Injectable({  
  providedIn: 'root'  
})
```

- A continuación:
 - En “productos.ts” se incluirá el servicio (eliminando el import a productos.ts que no se va a utilizar, y la asignación inicial de la variable productoSeleccionado).
 - Se definirá una variable para acceder al servicio.
 - Se accederá al método del servicio “getProductos()” para obtener el vector de productos.

```
...
import { ProductoServicio } from '../servicios/producto-servicio';
...
vProductos: Producto[] = [];
productoSeleccionado!:Producto;
constructor(private productoServicio: ProductoServicio ) { }
...

ngOnInit() {

    this.vProductos = this.productoServicio.getProductos();

}
...
```

- En nuestras aplicaciones podemos tener diferentes componentes gestionando las diferentes partes de nuestra vista. Por ejemplo, podemos tener una cabecera, una parte central y un pie de página, gestionados por diferentes componentes.

- Se van a utilizar una serie de iconos que contiene Material.
- Para poder usarlos debemos escribir esta línea en el fichero “styles.scss”:
 - `@import "https://fonts.googleapis.com/icon?family=Material+Icons";`
- Para utilizarlos, hemos de usar la etiqueta `<mat-icon>`:
 - `<mat-icon> home </mat-icon>`

Nota: Puede ser necesario reiniciar el servidor

- Creamos dos nuevos componentes para la cabecera y el pie de página.
 - `ng generate component cabecera`
 - `ng generate component pie`
- Se importarán estos componentes en `app.ts`
 - `import { Cabecera } from "../cabecera/cabecera";`
 - `import { Pie } from "../pie/pie";`
 - ...
 - `@Component({`
 - ...
 - `imports: [...CabeceraComponent, PieComponent]`
- Cambiaremos nuestra aplicación para que muestre esos componentes:
 - En el fichero `“app.html”`
 - `<app-cabecera></app-cabecera>`
 - `<app-productos></app-productos>`
 - `<app-pie></app-pie>`

- Modificar el contenido del .html y .scss de los componentes “cabecera” y “pie”, y de los estilos del fichero “styles.scss”.
- El fichero con el contenido a modificar se encuentra en el aula virtual (“código_html_estilos_angular_p3a.txt”).
- Importar los elementos de material necesarios en cabecera.ts
 - `import {MatToolbarModule} from '@angular/material/toolbar'`
 - `import { MatIconModule } from '@angular/material/icon';`
 - `import { MatIcon } from '@angular/material/icon';`
 - `@Component({`
 - `...`
 - `imports: [MatToolbarModule, MatIconModule, MatIcon],`

- Importar los elementos de material necesarios en pie.ts
 - `import {MatToolbarModule} from '@angular/material/toolbar'`
 - `import { MatIconModule } from '@angular/material/icon';`
 - `import { MatIcon } from '@angular/material/icon';`
 - `import { MatGridListModule } from '@angular/material/grid-list';`
 - `import { MatListModule } from '@angular/material/list';`
- `@Component({`
- `...`
- `imports: [MatToolbarModule, MatIconModule, MatIcon, MatGridListModule, MatListModule],`

[Inicio](#)  [Nosotros](#)  [Productos](#)  [Contacto](#)



Mi tienda

Precios baratos todo el año



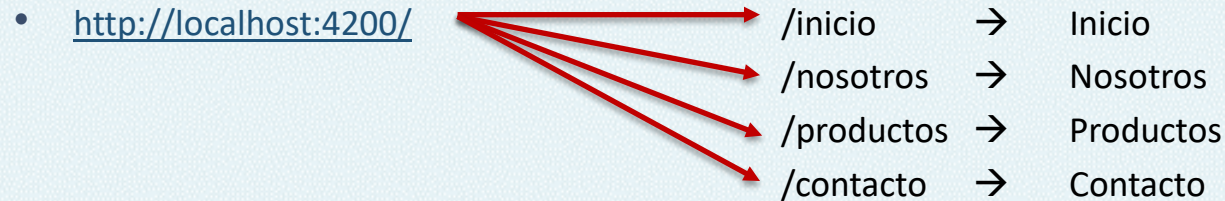
PRODUCTO 1 300	PRODUCTO 2 400	PRODUCTO 3 500	PRODUCTO 4 600
Links	Dirección		
 Inicio	<i>Calle nueva Burjassot, Valencia España</i>  : +555 555 5555  : +555 555 5555  : mitienda@moviles.net     		
 Nosotros			
 Productos			
 Contacto			

- Permite navegar entre los diferentes componentes de una aplicación, dando lugar a diferentes vistas.
- Se utilizan los enlaces o los botones de un menú, para navegar de una vista a otra.
- Al navegar se pueden pasar parámetros a las diferentes vistas.
- Para llevar a cabo esta navegación hace uso de instrucciones que permiten modificar la url en el navegador, sin necesidad de recargar toda la página.

- Se pueden utilizar las instrucciones que gestionan el historial de páginas visitadas de HTML5 para modificar una URL sin necesidad de recargar la página entera:
 - `pushState()`: Añade una entrada en el historial de navegación del navegador.
 - `replaceState()`: Modifica una entrada del historial.
 - Se puede pulsar los botones de atrás y adelante del navegador y no se recargarán las páginas desde el servidor.
 - Es necesario definir el elemento “base” indicando el path del cual no se llevará a cabo una recarga de la página.
 - `<base href="/">`
 - En angular siempre será en el directorio raíz.

Todas las páginas que se encuentren en ese directorio raíz serán manejadas por el módulo Route, sin necesidad de que el navegador realice una llamada al servidor para su descarga.

- Ejemplo:
- Podríamos cambiar entre las diferentes páginas, y no se recargaría la página desde el servidor



- Hay que definir Rutas para cada componente:
 - `{path: "/inicio", component: InicioComponent}`
- Para indicar que las vistas de una aplicación van a ser manejadas por el Módulo Router, se deberá incluir en la plantilla de la aplicación la directiva `routerOutlet`.
 - `<router-outlet></router-outlet>`
- Para gestionar la navegación entre las diferentes vistas, se utilizará la etiqueta `routerLink`.
 - `<a routerLink="/inicio"> Inicio </a>`

- Necesitamos disponer un componente para cada uno de los elementos del menú entre los que queremos intercambiar la vista.
 - Crear los componentes inicio, nosotros y contacto (productos ya está creado).
 - Utilizar el comando: `ng generate component nombre_del_Componente`
- En nuestra aplicación ya está importado el módulo Router, en `app.ts`:
 - `import { RouterOutlet } from '@angular/router';`
 - ...
 - **imports:** [
 RouterOutlet, ..
 -],

- Las rutas de la aplicación se definirán en el fichero, “app.routes.ts”
 - Ya se encuentra importado el módulo de rutas

```
import { Routes } from '@angular/router';
```
 - Hay que importar los diferentes componentes entre los que vamos a navegar

```
import { Productos } from './productos/productos';  
import { DetalleProducto } from './detalle-producto/detalle-producto';  
import { Inicio } from './inicio/inicio';  
import { Nosotros } from './nosotros/nosotros';  
import { Contacto } from './contacto/contacto';
```
 - Se definen las rutas
 - Se especifica el “path” y el “componente” al que se visualizará al navegar a ese path. (Ejemplo: ["http://localhost:4200/inicio"](http://localhost:4200/inicio))

```
export const routes: Routes = [  
  { path: 'inicio', component: Inicio },  
  { path: 'productos', component: Productos },  
  { path: '', redirectTo: '/inicio', pathMatch: 'full' }  
];
```
 - El último path, se corresponde con la url ["http://localhost:4200/"](http://localhost:4200/), que por defecto redirigirá a la página “inicio”

- El fichero de rutas se encuentra importado en el fichero “app.config.ts”
 - `import { routes } from './app.routes';`

- Para hacer uso de las rutas se utiliza la etiqueta `<router-outlet>` en la plantilla de visualización de la aplicación.

- En “app.html” quedará:

```
<app-cabecera></app-cabecera>
<router-outlet></router-outlet>
<app-pie></app-pie>
```

- actualizar los enlaces de la cabecera, utilizando el atributo “routerLink”.
- En “cabecera.html”

```
<a mat-button routerLink="/inicio"><mat-icon>home</mat-icon> Inicio </a>&nbsp;
<a mat-button><mat-icon>info</mat-icon> Nosotros </a>&nbsp;
<a mat-button routerLink="/productos"><mat-icon>list</mat-icon> Productos </a>&nbsp;
<a mat-button><mat-icon>contact_mail</mat-icon> Contacto </a>
```

- Añadir el import “RouterLink” En “cabecera.ts”

- import { RouterLink } from "@angular/router";
 - ...
 - @Component({
 - ...
 - imports: [...,RouterLink]

- Establecer el contenido, implementar las rutas y redefinir los enlaces para las opciones del menú: nosotros y contacto.
- El diseño del contenido de esos apartados se encuentra en el aula virtual, “codigo_html_estilos_angular_p3b.txt”.

[Inicio](#)  [Nosotros](#)  [Productos](#)  [Contacto](#)



Mi tienda

Precios baratos todo el año

Nosotros

Nuestra Historia

Somos una empresa valenciana fundada en abril de 2001 para vender teléfonos móviles libres por Internet. Nacimos justo después de la burbuja de las .com para liberar a los usuarios de telefonía móvil de los móviles bloqueados por las operadoras, permitiendo que cualquier persona pudiera cambiar libremente de compañía ...

Personal

Mostrar el personal aquí

Links	Dirección
 Inicio	Calle nueva Burjassot, Valencia España  : +555 555 5555  : +555 555 5555  : mitienda@moviles.net
 Nosotros	
 Productos	
 Contacto	

© Copyright 2024 Mi tienda

[Inicio](#)  [Nosotros](#)  [Productos](#)  [Contacto](#)



Mi tienda

Precios baratos todo el año

Contacto

Localización

Dirección

Calle nueva
Burjassot, Valencia
España

 : +555 555 5555
 : +555 555 5555
 : mitienda@moviles.net

 [Llamar](#)  [Email](#)

Mapa

Links	Dirección
 Inicio	Calle nueva Burjassot, Valencia España
 Nosotros	
 Productos	 : +555 555 5555  : +555 555 5555  : mitienda@moviles.net
 Contacto	

- Añadir una nueva propiedad “oferta” de tipo boolean a la clase “productos” (“producto.ts”) que servirá para remarcar los productos en oferta en la página Inicial.
- Añadir el campo a los datos de los productos (“productosBD.ts”).
- Poner un producto en oferta.
- Añadir dos funciones en el servicio productoServicio (“producto-servicio.ts”)
 - Obtener un producto por su id.

```
getProducto(id: number): Producto {  
    return PRODUCTOS.filter((producto) => (producto.id === id))[0];  
}
```
 - Obtener los productos en oferta.

```
getProductosOferta(): Producto[] {  
    return PRODUCTOS.filter((producto) => producto.oferta);  
}
```

- Ahora mostraremos los productos de oferta en la plantilla de visualización del componente “inicio”.
- Primero se importan la clase “Producto” y el servicio “ProductoService” en el fichero “inicio.ts”.
- Se pone el servicio como parámetro del constructor.
- Se crea una variable para almacenar los productos en oferta, obteniendo esos productos con la función del servicio “getProductosOferta”.
- Ahora se mostrarán en la pantalla de inicio los productos en oferta.
- (“codigo_html_estilos_angular_p3c.txt”).

- Con la propiedad “routerLinkActive”, se pueden especificar estilos para aquel enlace que se encuentra activo actualmente en la página.
- En la cabecera se puede utilizar en cada enlace del menú (cabecera.html).
routerLinkActive="activo"
- Después se puede definir un estilo “activo” para el enlace que se encuentre activo en el fichero “cabecera.scss”:

```
...  
$background-moredark: #4527A0;  
...  
.activo {  
    background: $background-moredark;  
}
```

- Añadir en cabecera.ts, el import correspondiente:
import { RouterLinkActive } from "@angular/router";
...
imports: [..., RouterLinkActive],

- Página Web de Angular
 - <https://angular.io/>
- Página Web de Angular Servicios
 - <https://angular.io/guide/architecture#services>
- Página Web de Angular Routing
 - <https://angular.io/guide/router>



VNIVERSITAT
DE VALÈNCIA